

Problem 1 : The Last Astronaut

Pseudocode :

ALGORITHM LastAstronautSimulation

INPUT:

N : Integer (Jumlah total astronot)

K : Integer (Nilai hitungan eliminasi awal)

OUTPUT:

Urutan astronot yang dieliminasi dan nomor astronot yang bertahan.

BEGIN

// 1. Inisialisasi daftar astronot dari nomor 1 hingga N

AstronautList $\leftarrow$ ALLOCATE ARRAY OF SIZE N

FOR i FROM 0 TO N - 1 DO

AstronautList[i] $\leftarrow$ i + 1

ENDFOR

// 2. Inisialisasi posisi indeks awal perhitungan

CurrentIndex $\leftarrow$ 0

PRINT "Urutan astronot yang dieliminasi: "

// 3. Proses eliminasi berulang hingga tersisa 1 astronot

WHILE SIZE(AstronautList) > 1 DO

// Hitung posisi indeks astronot yang akan dieliminasi memakai operasi modulo

EliminateIndex $\leftarrow$ (CurrentIndex + K - 1) MOD SIZE(AstronautList)

```

// Ambil nomor astronot yang berada di indeks tersebut
EliminatedAstronaut ← AstronautList[EliminateIndex]
PRINT EliminatedAstronaut + " "

// Hapus astronot yang terpilih dari daftar
REMOVE_ELEMENT(AstronautList, EliminateIndex)

// 4. Aturan perubahan dinamis untuk nilai K
IF EliminatedAstronaut MOD 2 == 0 THEN
    K ← K + 2
ELSE
    K ← K - 1
ENDIF

// 5. Validasi batas bawah nilai K (K tidak boleh kurang dari 2)
IF K < 2 THEN
    K ← 2
ENDIF

// Indeks berikutnya otomatis bergeser ke posisi astronot setelahnya
CurrentIndex ← EliminateIndex

ENDWHILE

// 6. Output astronot terakhir yang berhasil bertahan
PRINT "Astronot terakhir yang bertahan: " + AstronautList[0]

```

END

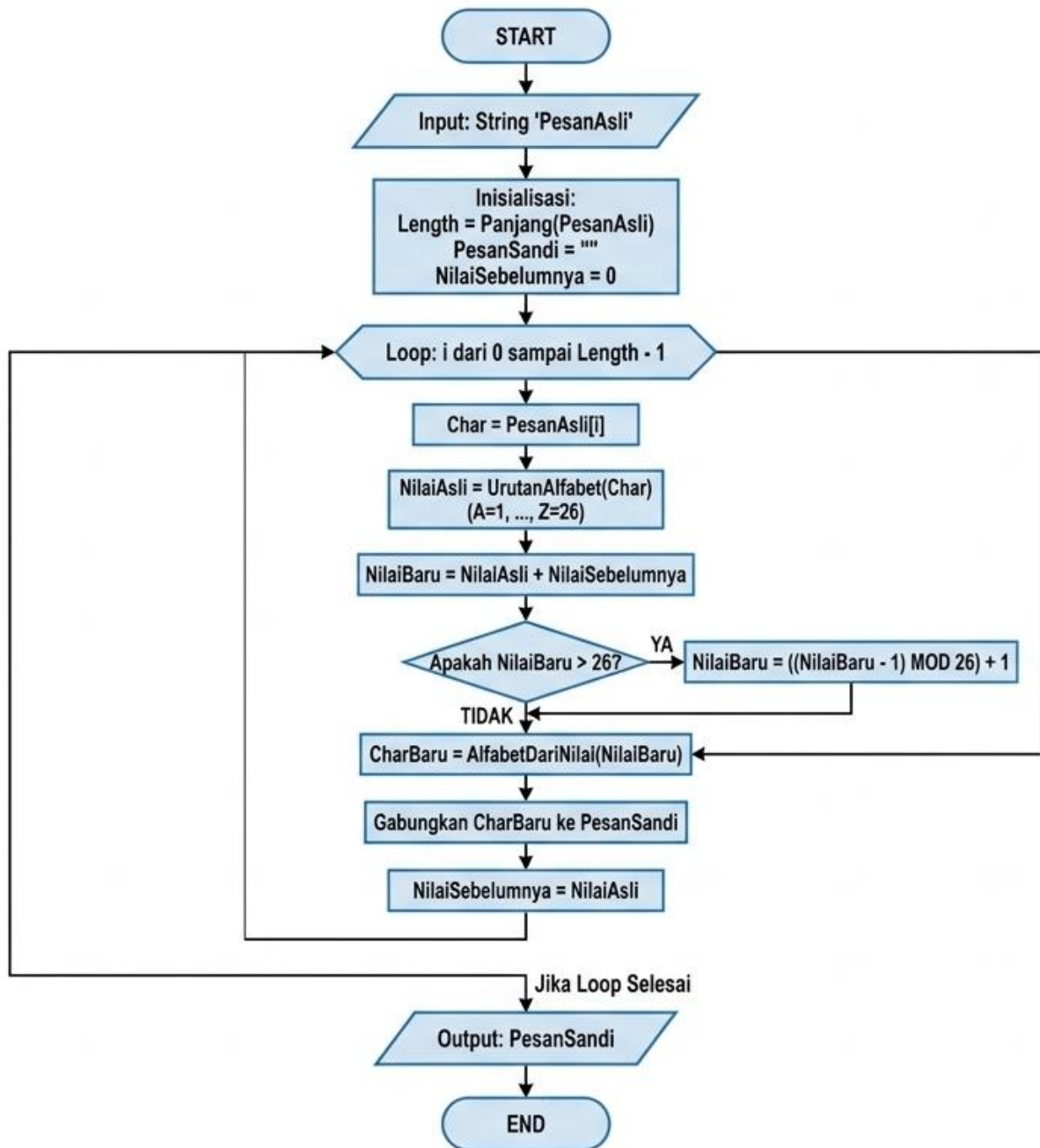
Penjelasan Logika :

Logika penyelesaian masalah *The Last Astronaut* ini mengadaptasi variasi dari permasalahan klasik *Josephus Problem*, di mana sekumpulan objek dalam formasi lingkaran dieliminasi secara periodik dengan langkah tertentu. Algoritma di atas memanfaatkan struktur data dinamis berupa *List* atau *Array* untuk menampung nomor urut unik setiap astronot dari 1 hingga N. Untuk mensimulasikan pergerakan melingkar secara efisien tanpa melakukan iterasi manual satu per satu, algoritma menggunakan operasi aritmatika sisa bagi atau *modulo* (MOD). Indeks astronot yang akan dieliminasi ditentukan melalui rumus target indeks yang disesuaikan dengan panjang ukuran *array* yang tersisa pada saat itu.

Setelah menemukan target astronot, program akan mencatat nomor uniknya sebagai bagian dari urutan eliminasi, lalu menghapusnya dari struktur data. Setelah penghapusan berhasil dilakukan, algoritma akan mengevaluasi nomor unik astronot tersebut melalui pengecekan kondisi (IF-ELSE). Jika nomornya genap, nilai langkah hitungan K ditambahkan dengan 2. Jika ganjil, nilai K dikurangi 1. Algoritma juga dilengkapi dengan fungsi pengaman untuk memastikan nilai K tidak pernah berada di bawah angka 2. Proses ini diulang secara terus menerus dalam sebuah perulangan (*looping*) hingga ukuran *array* menyusut dan menyisakan tepat satu elemen terakhir, yang dinobatkan sebagai astronot yang bertahan.

Problem 2 : Alien-In-The-Middle

Flowchart :



Penjelasan Logika :

Sistem sandi **Alien-In-The-Middle** menggunakan metode pergeseran huruf dinamis yang bergantung pada nilai alfabet dari huruf tepat di sebelumnya. Proses dimulai dengan membiarkan huruf pertama pada pesan asli tetap sama tanpa perubahan apa pun. Untuk huruf-huruf berikutnya, posisinya akan digeser maju sebanyak nilai dari karakter yang berada langsung di kirinya. Nilai pergeseran ini dihitung berdasarkan urutan alfabet standar, di mana **huruf A bernilai 1, B bernilai 2, hingga Z bernilai 26**. Sebagai contoh pada kata "ALIENS", huruf pertama 'A' tetap menjadi 'A'. Huruf kedua 'L' (nilai 12) digeser sebanyak nilai huruf sebelumnya yaitu 'A' (1), sehingga berubah menjadi 'M' (13).

Tantangan utama dari algoritma ini terletak pada penanganan pergeseran yang melebihi batas akhir alfabet. Jika hasil penjumlahan posisi huruf saat ini dengan nilai huruf sebelumnya melewati angka 26 (huruf Z), maka perhitungan posisinya harus diputar kembali (**wrapping**) mulai dari huruf A. Secara matematis, kondisi ini dapat diselesaikan menggunakan operasi modulus 26 untuk memastikan indeks karakter baru selalu berada di rentang 1 sampai 26. Proses ini dilakukan secara berurutan satu per satu dari kiri ke kanan hingga akhir kata, sehingga menghasilkan sebuah teks tersandi baru—seperti kata "ALIENS" yang berubah menjadi "**AMUNSG**"—yang siap digunakan untuk mengelabui sistem komunikasi tersebut.